PS

Dataset: Kaggle - NSL-KDD Dataset-https://www.kaggle.com/datasets/hassan06/nslkdd
Baseline Paper: "A Detailed Analysis of the KDD CUP 99 Data Set" (IEEE CISDA).

Group Assignments & Search Frameworks:

Group 1: Random Forest Classifier ("An Ensemble Random Forest Approach for Intrusion Detection System Using NSL-KDD Dataset")Target Search Framework: "An Ensemble Random Forest Approach for Intrusion Detection System Using NSL-KDD Dataset" (IEEE Layout Style).Implementation Mechanics: Implement a Random Forest classifier using sklearn.ensemble. Run parameter grid sweeps across tree depths and splits (max_depth=5 to 25, criterion="gini" vs "entropy"). Output explicit Feature Importance charts using tree impurity decay.

Group 2: Support Vector Machine ("Performance Evaluation of Support Vector Machine (SVM) Kernels on NSL-KDD Dataset") Target Search Framework: "Performance Evaluation of Support Vector Machine (SVM) Kernels on NSL-KDD Dataset" (IEEE Layout Style).Implementation Mechanics: Build a Support Vector Machine via sklearn.svm.SVC. Compare performance metrics using Linear vs. Radial Basis Function (RBF) kernels. Document mathematical margin equations and map the classification boundaries over the highest-correlated feature pairs.

Group 3 (Data Lead): LightGBM Classifier ("Network Intrusion Detection System Using Boosting Classifiers: LightGBM and XGBoost") Target Search Framework: "Network Intrusion Detection System Using Boosting Classifiers: LightGBM and XGBoost" (Springer Layout Style).Implementation Mechanics: Deploy a leaf-wise LightGBM classifier via the lightgbm library. Optimize early stopping iterations to stop overtraining on minor network anomalies. Plot log-loss reduction curves tracking optimization progress over successive tree growth rings.

Group 4: Extreme Gradient Boosting (XGBoost) Framework. Target Search Framework: "Extreme Gradient Boosting (XGBoost) Framework for High-Dimensional Network Intrusion Filters" (IEEE Layout Style).Implementation Mechanics: Configure an Extreme Gradient Boosting model via the xgboost framework. Tune tree regularizations (alpha, lambda) to optimize system resilience against class imbalances. Compute a multi-leaf breakdown tracking structural weights across continuous numeric columns.

Group 5: Multi-Layer Perceptron ("Multilayer Perceptron Neural Network Architecture for Network Anomaly Detection") Target Search Framework: "Multilayer Perceptron Neural Network Architecture for Network Anomaly Detection" (ACM Layout Style).Implementation Mechanics: Code a 4-layer feed-forward Multilayer Perceptron via sklearn.neural_network.MLPClassifier. Layer dimensions must scale as [64 -> 32 -> 16]. Use an adaptive Adam optimizer with a custom learning rate decay table, and graph the loss curve over epochs.

Theme: Network Intrusion Detection (IDS)

Group 5: Multi-Layer Perceptron ("Multilayer Perceptron Neural Network Architecture for Network Anomaly Detection") Target Search Framework: "Multilayer Perceptron Neural Network Architecture for Network Anomaly Detection" (ACM Layout Style). Implementation Mechanics: Code a 4-layer feed-forward Multilayer Perceptron via sklearn.neural_network.MLPClassifier. Layer dimensions must scale as [64 -> 32 -> 16]. Use an adaptive Adam optimizer with a custom learning rate decay table, and graph the loss curve over epochs.

WHAT WE HAVE TO DO

Recommended paper structure

1. Title, authors, abstract

Use the ACM two-column layout with:

- Title: *Multilayer Perceptron Neural Network Architecture for Network Anomaly Detection*
- Group number, member names, course, institution, date
- Abstract: 150–250 words covering the problem, dataset, 4-layer MLP, Adam optimizer with learning-rate decay, main metrics, and key finding.
- Keywords: Network Anomaly Detection, Intrusion Detection System, Multilayer Perceptron, NSL-KDD, Supervised Learning, Adam Optimizer.

2. Introduction

Explain why anomaly detection matters: modern networks face DoS, probing, unauthorized access, and other attacks, and manual monitoring cannot scale. State that you evaluate a supervised MLP on NSL-KDD to distinguish normal traffic from attack traffic.

End the section with:

- **Problem statement:** Can a compact 4-layer MLP accurately detect anomalous network connections in NSL-KDD?
- **Objective:** Build, train, and evaluate an MLP-based anomaly detector.
- **Contributions:** a reproducible preprocessing pipeline, a [64 → 32 → 16] hidden-layer architecture, Adam-based training with decay, and an empirical evaluation using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrix.

3. Related Work

Use your baseline paper, *"A Detailed Analysis of the KDD CUP 99 Data Set"* (IEEE CISDA), as the foundation. It explains limitations of KDD Cup '99—such as redundancy and biased records—and motivates NSL-KDD as an improved benchmark.torontomu+1

Cover briefly:

- KDD Cup '99 and its limitations.
- NSL-KDD improvements: reduced redundancy and more reasonable training/test difficulty.redalyc
- Prior MLP/neural-network intrusion-detection studies.
- Why MLPs are suitable: they learn nonlinear relationships among connection features.

4. Dataset Description

Describe NSL-KDD clearly:

Item	What to include
Source	Kaggle NSL-KDD dataset.
Records	Separate `KDDTrain+` and `KDDTest+` files.
Features	41 input features plus the class label. arxiv+1
Feature groups	Basic TCP/IP connection features, content features, and traffic/host-based features. arxiv
Labels	Normal traffic and attack categories: DoS, Probe, R2L, U2R. link.springer+1
Task setting	Use binary classification: `normal` vs. `anomaly`, unless your instructor requires 5-class classification.

Include a table of class distribution before and after preprocessing. This is important because attack types are imbalanced, especially U2R and R2L.

5. Methodology

This is the core section. Present it as a pipeline:

Raw NSL-KDD → Cleaning → Encoding → Scaling → Train/Test Split → MLP Training → Evaluation → Loss Curve Analysis

5.1 Data preprocessing

Include:

- Remove missing or malformed rows, if any.
- One-hot encode categorical features: `protocol_type`, `service`, `flag`.
- Encode labels: `normal = 0`, `attack = 1` for binary anomaly detection.
- Apply `StandardScaler` to numerical features; fit the scaler only on training data.
- Use the provided train/test split rather than randomly splitting everything, because NSL-KDD is designed with separate train and test sets.redalyc

Mention class imbalance and state whether you use `class_weight`, oversampling, or simply report per-class metrics. For an academic paper, reporting per-class precision/recall is better than only overall accuracy.

5.2 MLP architecture

Specify the exact network:

Component	Configuration
Input layer	Number of features after one-hot encoding and scaling
Hidden layer 1	64 neurons, ReLU
Hidden layer 2	32 neurons, ReLU
Hidden layer 3	16 neurons, ReLU
Output layer	1 neuron with sigmoid probability, or 2-class softmax
Total depth	4 layers including output layer

In `MLPClassifier`, this corresponds to:

python
```
hidden_layer_sizes=(64, 32, 16)
activation='relu'
```

Explain why the architecture narrows: it progressively compresses high-dimensional network-traffic features into a smaller representation, encouraging the model to learn generalized attack patterns rather than memorizing individual records.

5.3 Optimizer and learning-rate decay

Use Adam and describe the decay table explicitly. Scikit-learn's `MLPClassifier` supports `solver='adam'`; its `adaptive` schedule keeps the initial learning rate while loss improves and divides it by 5 after consecutive non-improving epochs.[sklearn]

A suitable custom decay table could be:

Epoch range	Learning rate
1–20	0.001
21–40	0.0005
41–60	0.00025
61–80	0.0001
81–100	0.00005

Important implementation note: standard `MLPClassifier` does not directly accept an epoch-wise learning-rate table. To satisfy the requirement, either:

- use `learning_rate='adaptive'` and document that as the adaptive decay mechanism, or
- implement a small custom training loop using `MLPClassifier` with `warm_start=True`, changing `learning_rate_init` at epoch checkpoints.

The second approach best matches the phrase “custom learning rate decay table.”

Example:

python

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, roc_auc_score, confusion_matrix, classification_report
import numpy as np
import matplotlib.pyplot as plt

decay_table = {
    0: 0.001,
    20: 0.0005,
    40: 0.00025,
    60: 0.0001,
    80: 0.00005
}

mlp = MLPClassifier(
    hidden_layer_sizes=(64, 32, 16),
    activation='relu',
    solver='adam',
    learning_rate_init=0.001,
    max_iter=1,
    warm_start=True,
    random_state=42
)

loss_history = []

for epoch in range(100):
    if epoch in decay_table:
        mlp.learning_rate_init = decay_table[epoch]
    mlp.fit(X_train, y_train)
    loss_history.append(mlp.loss_)

plt.plot(range(1, len(loss_history) + 1), loss_history)
```

```
plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("MLP Training Loss Curve on NSL-KDD")
plt.grid(True)
plt.savefig("loss_curve.png", dpi=300)
```

6. Experimental Setup

State every experimental detail so someone can reproduce your work:

- Programming language: Python.
- Libraries: `pandas`, `scikit-learn`, `matplotlib`, `seaborn`, optionally `imbalanced-learn`.
- Train/test data: NSL-KDD `KDDTrain+` and `KDDTest+`.
- Random seed: e.g., 42.
- Epochs: 100.
- Batch size: default or explicitly set, e.g., 128.
- Optimizer: Adam.
- Learning-rate schedule: the table above.
- Activation: ReLU for hidden layers.
- Output activation: sigmoid/softmax depending on implementation.
- Evaluation strategy: binary anomaly detection, with optional attack-category analysis.

7. Results and Discussion

Report both tables and figures.

Required tables

1. **Dataset summary:** train/test size, number of features, class distribution.
2. **MLP configuration:** layers, neurons, activation, optimizer, learning-rate schedule, epochs, batch size.
3. **Performance metrics:**

Metric	What it measures
Accuracy	Overall correct predictions
Precision	How many predicted attacks are actually attacks
Recall	How many actual attacks the model detects
F1-score	Balance of precision and recall
ROC-AUC	Ranking quality across thresholds

Training time	Practical feasibility

4. **Per-class results:** Normal, DoS, Probe, R2L, U2R—if you perform multiclass evaluation.
5. **Comparison with baseline:** Compare your MLP against at least one simple baseline such as Logistic Regression, Decision Tree, or a default `MLPClassifier`.

Required figures

- **Loss curve over epochs:** mandatory.
- **Learning-rate decay curve:** helpful and directly supports your custom decay table.
- **Confusion matrix:** shows normal vs. anomaly errors.
- **ROC curve:** especially useful for binary anomaly detection.
- **Class-wise precision/recall bar chart:** useful because NSL-KDD is imbalanced.
- **Architecture diagram:** input layer → 64 → 32 → 16 → output.

8. Discussion Points

Interpret results rather than merely reporting them:

- Did the loss decrease smoothly? A sharp initial drop followed by gradual stabilization is expected.
- Did lowering the learning rate improve stability or reduce overfitting?
- Which attack classes were hardest to detect? Usually R2L and U2R are difficult because they have fewer samples.riverpublishers
- Was recall high enough for a security system? In anomaly detection, missing an attack is often more costly than generating a false alarm.
- What are the limitations? NSL-KDD is based on older network traffic and does not fully represent modern network environments.eprints.ugd.edu

9. Conclusion and Future Work

State whether the 4-layer MLP met the objective. Then propose realistic extensions:

- Feature selection using correlation, mutual information, or PCA.
- SMOTE or class weighting for minority attacks.
- Hyperparameter tuning with grid search or Bayesian optimization.
- Comparison with Random Forest, XGBoost, CNN, or autoencoder-based anomaly detection.
- Deployment-oriented evaluation using latency and memory footprint.

Suggested ACM sections

Use these headings in the final paper:

1. Introduction

2. Related Work
3. Dataset and Problem Formulation
4. Proposed MLP-Based Anomaly Detection Framework
5. Experimental Setup
6. Results and Discussion
7. Conclusion and Future Work
8. References

Minimum deliverables

For full marks, your submission should include:

- ACM-formatted PDF paper.
- Python notebook or `.py` script.
- Preprocessing pipeline.
- Trained 4-layer MLP using `(64, 32, 16)`.
- Adam optimizer with documented learning-rate decay.
- Loss-versus-epoch graph.
- Confusion matrix and classification report.
- ROC-AUC curve.
- Results table comparing your model with a baseline.
- Reproducibility details: random seed, library versions, train/test files.

Vrinda Content

1. Fundamentals of the project

1.1 What is an Intrusion Detection System (IDS)?

An Intrusion Detection System (IDS) is a security system that monitors network traffic or computer activity to identify signs of malicious behavior, unauthorized access, or violations of security policies.

An IDS typically observes activity, analyzes it, and generates an alert when suspicious behavior is detected. Unlike an Intrusion Prevention System (IPS), an IDS is primarily designed to detect and report suspicious activity rather than automatically block it.

There are two common approaches:

- Signature-based detection: Looks for known attack patterns or signatures. It can be effective for recognized threats but may miss new or modified attacks.
- Anomaly-based detection: Learns or defines what normal activity looks like and identifies activity that deviates from that baseline. It can potentially detect previously unseen attacks, but unusual legitimate activity may also trigger false alarms.

Machine learning can help an anomaly-based IDS classify network connections as normal or suspicious based on their observed characteristics.

1.2 What is network anomaly detection?

Network anomaly detection is the process of identifying network activity that differs from expected or normal behavior.

Examples include:

- An unusually high number of connection requests, potentially indicating a denial-of-service attack.
- Repeated connection attempts that may indicate probing or reconnaissance.
- Unexpected traffic patterns or access behavior.
- Network connections with characteristics associated with known attack classes.

An anomaly is not automatically an attack. Legitimate network changes can also produce unusual traffic. Therefore, a detection model must be evaluated for both its ability to detect attacks and its tendency to incorrectly flag normal connections.

In this project, network anomaly detection is treated as a supervised binary classification task: each connection is classified as either normal or attack using labeled NSL-KDD data.

1.3 What is a Multilayer Perceptron (MLP)?

A Multilayer Perceptron is a feedforward artificial neural network made up of interconnected neurons arranged in layers. It learns relationships between input features and target labels by adjusting its weights and biases during training.

The main components are:

- Input layer: Receives the numerical representation of network connection features.
- Hidden layers: Learn patterns and relationships in the input data.
- Output layer: Produces the model's prediction, such as normal or attack.
- Weights and biases: Trainable parameters used to calculate predictions.
- Activation functions: Introduce nonlinearity so the network can learn complex patterns.

For this project, the planned hidden-layer architecture is:

Input layer

Preprocessed network connection features

Hidden layer 1 — 64 neurons

ReLU activation

Hidden layer 2 — 32 neurons

ReLU activation

Hidden layer 3 — 16 neurons

ReLU activation

Output layer

Probability of attack; classification into normal or attack

This is the proposed architecture, not evidence that it is better than other architectures. The input size depends on the number of features remaining after preprocessing. The exact output configuration depends on Member 3's implementation; for example, a one-unit sigmoid output can be used in a neural-network implementation, while scikit-learn's `MLPClassifier` uses its own output-layer configuration for binary classification.

1.4 What is the ReLU activation function?

The Rectified Linear Unit (ReLU) is commonly defined as:

$$f(x) = \max(0, x)$$

It outputs zero for negative input values and the input value itself for positive values.

ReLU is widely used in hidden layers because it is computationally simple and helps neural networks learn nonlinear relationships. However, some neurons can become inactive for many inputs, a problem commonly called the dying ReLU issue.

1.5 What is the Adam optimizer?

Adam stands for Adaptive Moment Estimation. It is an optimization algorithm that updates neural-network parameters using estimates of the first and second moments of gradients.

In simpler terms, Adam adjusts the model's weights during training to reduce the prediction loss. It adapts the effective step size for individual parameters, which often makes it a convenient optimizer for neural-network training.

Adam does not guarantee the best possible result for every dataset. Its performance depends on factors such as the learning rate, model architecture, preprocessing, and training settings.

1.6 What is learning-rate decay?

The learning rate controls how large the optimizer's parameter updates are during training.

- A high learning rate can make training progress quickly, but updates may be unstable.
- A low learning rate can produce smaller updates, but training may progress slowly.
- Learning-rate decay reduces the learning rate during training to allow progressively smaller updates.

Your project proposes a custom epoch-wise learning-rate schedule. One possible design is to start with a learning rate of 0.001 and reduce it at predetermined epochs. The exact schedule must be specified and tested by Member 3.

Important implementation detail: `sklearn.neural_network.MLPClassifier` supports `solver="adam"` and a fixed `learning_rate_init`, but its `learning_rate` schedule setting applies to the SGD solver rather than providing a general epoch-wise decay schedule for Adam. Therefore, a genuinely custom Adam decay schedule may require an implementation that exposes optimizer updates, or a carefully documented alternative. Do not claim that a custom Adam schedule was used with standard `MLPClassifier` unless the implementation actually supports it.

2. NSL-KDD dataset: background and description

2.1 What is NSL-KDD?

NSL-KDD is a benchmark dataset used to evaluate machine-learning approaches for network intrusion detection. It was introduced by Tavallaee and colleagues in 2009 to address important statistical problems in the older KDD Cup 1999 dataset, particularly the presence of many redundant records.

The dataset represents network connections as structured records. Each record contains 41 input features and a label. The features describe connection characteristics, such as protocol, service, connection duration, traffic volume, and indicators related to suspicious activity.

2.2 Dataset files used in your project

File	Purpose	Number of records
`KDDTrain+`	Train the model	125,973
`KDDTest+`	Evaluate the trained model on test data	22,544

These are the commonly reported record counts for the standard NSL-KDD files.

The team should train the model on `KDDTrain+` and use `KDDTest+` for final evaluation. The test set should not be used to fit preprocessing transformations, select hyperparameters, or make repeated decisions about which model to keep. A separate validation set or validation split from the training data should be used for those decisions.

2.3 What types of attacks are represented?

The original NSL-KDD labels include normal connections and four broad attack families:

Category	Meaning	Example of the type of behavior
DoS	Denial of Service	Attempts to disrupt service availability
Probe	Reconnaissance or probing	Scanning hosts, ports, or services
R2L	Remote to Local	Attempts to gain local access remotely
U2R	User to Root	Attempts to obtain elevated privileges

These categories describe the original multiclass label structure.

For your team's binary classification experiment, the labels are combined as follows:

- `normal` → Normal
- Any attack label, including DoS, Probe, R2L, and U2R attacks → Attack

The team must implement this mapping consistently in both training and test data. It should document how labels are normalized and how the binary classes are encoded.

2.4 What are the 41 features?

The 41 features are conventionally grouped into three broad types:

1. Basic connection features: Describe individual connections, including duration, protocol, service, connection status, and byte counts.
2. Content features: Describe properties associated with connection content or behavior, such as failed login attempts and login status.
3. Traffic features: Describe connection patterns involving a host or service over a defined period or window.

The original dataset has three categorical features: `protocol_type`, `service`, and `flag`. These cannot simply be treated as ordinary continuous numbers without considering their meaning.

2.5 Advantages of NSL-KDD

- It removes redundant records that affected the original KDD Cup 1999 dataset.
- Its size is manageable for standard machine-learning experiments.
- It provides labeled normal and attack examples.
- It enables comparison with a substantial body of previous intrusion-detection research.
- Its test set includes attack patterns that are not represented in the training set, making generalization an important consideration.

2.6 Limitations of NSL-KDD

NSL-KDD is useful for reproducible benchmark experiments, but it should not be treated as a perfect representation of modern network traffic.

- It is derived from older traffic and attack scenarios, so it does not fully represent today's cloud, mobile, IoT, encrypted, and rapidly evolving network environments.
- Its records are engineered connection features rather than a complete, contemporary live traffic stream.
- Performance on NSL-KDD does not prove that a model will perform equally well in a real operational network.
- Rare attack types can be difficult to detect, so overall accuracy alone can hide poor attack detection.

- Results may depend heavily on preprocessing, label mapping, class distribution, and evaluation methodology.

Conclusion about the dataset: NSL-KDD is appropriate for a controlled educational comparison of an MLP and a baseline classifier. The paper should state that it is a benchmark study and avoid claiming real-world effectiveness solely from results on this dataset.

3. Literature review: important research papers

Below are ten relevant publications to begin with. The summaries distinguish each paper's contribution from conclusions your team still needs to test. The limitation column identifies a limitation of the paper's scope or relevance to your project, not necessarily a flaw in the original authors' work.

Paper	Main contribution and relevance	Limitation or relevance to your project
Denning, 1987 — An Intrusion-Detection Model	Introduces a foundational model for intrusion detection based on observable system behavior and profiles. It helps explain the basic concept of anomaly detection.	It predates modern deep learning and does not evaluate your proposed MLP architecture.
McHugh, 2000 — Testing Intrusion Detection Systems: A Critique of the 1998 and 1999 DARPA Intrusion Detection System Evaluations	Critically examines how intrusion-detection systems and their evaluation datasets should be assessed. It supports the argument that evaluation design affects the credibility of results.	It is primarily about evaluation methodology, not the performance of your specific neural network.
Tavallaee et al., 2009 — A Detailed Analysis of the KDD CUP 99 Data Set	Analyzes issues in KDD Cup 1999 and introduces NSL-KDD as a revised benchmark. This is the essential reference for your dataset choice.	NSL-KDD still inherits limitations from an older traffic-generation environment.

Sommer and Paxson, 2010 — Outside the Closed World: On Using Machine Learning for Network Intrusion Detection	Explains why machine-learning approaches to network security face challenges when real-world conditions differ from training assumptions.	It is a conceptual and methodological discussion, not a direct test of your proposed 64–32–16 architecture.
Patcha and Park, 2007 — An Overview of Anomaly Detection Techniques: Existing Solutions and Latest Technological Trends	Reviews anomaly-detection techniques and their applications, providing background for the motivation behind anomaly-based detection.	It predates many modern deep-learning approaches and does not establish the best architecture for NSL-KDD.
Moustafa and Slay, 2015 — UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems	Introduces the UNSW-NB15 dataset and provides context for the need to evaluate intrusion detection on datasets beyond older benchmarks.	It concerns a different dataset; its results cannot be transferred directly to NSL-KDD.
Shone et al., 2018 — A Deep Learning Approach to Network Intrusion Detection	Investigates deep-learning techniques for network intrusion detection and provides context for learned feature representations.	Its architecture is not the same as the proposed scikit-learn MLP, so direct performance comparisons require matching experimental conditions.
Moustafa and Slay, 2016 — The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set	Studies dataset characteristics and the implications of choosing an intrusion-detection benchmark.	Its focus is on dataset evaluation and comparison, rather than your particular MLP and learning-rate schedule.

A Survey of Neural Networks Usage for Intrusion Detection Systems, 2021	Reviews neural-network approaches to intrusion detection and discusses datasets and model families used in this field.	A broad survey cannot replace an experiment that controls architecture, optimizer, preprocessing, and test data.
A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks, IEEE publication	Examines recurrent neural networks for intrusion detection and discusses NSL-KDD, feature encoding, and test-set evaluation.	Recurrent networks differ from feedforward MLPs, so the paper is useful for comparison and context rather than as an exact reproduction.

The publications and their relevance are supported by the original dataset paper, foundational evaluation literature, and scholarly reviews of anomaly detection and neural-network intrusion detection.

3.1 What should you write after reviewing these papers?

The central message from this literature is that machine learning can help identify malicious network activity, but results depend on dataset quality, preprocessing, class distribution, the model selected, and the evaluation method.

Traditional anomaly-detection work establishes the problem and the challenges of identifying deviations from normal behavior. Dataset research highlights the danger of drawing conclusions from unsuitable or redundant benchmark records. Neural-network research explores learned representations and nonlinear classification, while surveys reveal that many model families and evaluation approaches are used.

Your study builds on this work by testing a clearly specified MLP on NSL-KDD and comparing it with a simpler baseline under the same data and evaluation conditions. The contribution is an empirical comparison, not a claim that an MLP is universally superior.

4. Ready-to-use Introduction for the paper

The following is a draft your team can adapt for its ACM-style paper. It should be reviewed alongside the team's actual implementation and results.

1. Introduction

Computer networks are essential to modern communication, information sharing, and digital services. However, networked systems face security threats such as denial-of-service attacks, reconnaissance, and attempts to gain unauthorized access. Intrusion Detection Systems (IDSs) help identify potentially malicious activities by monitoring network behavior and analyzing relevant traffic characteristics.

Traditional signature-based detection methods identify attacks using previously recognized patterns. Although useful for known threats, these methods may have difficulty identifying previously unseen or modified attacks. Anomaly-based detection offers an alternative by identifying behavior that differs from expected patterns. Machine-learning methods can support this process by learning relationships between network connection features and their associated labels.

A Multilayer Perceptron (MLP) is a feedforward neural network capable of learning nonlinear relationships between input features and output classes. Its use in network intrusion detection is motivated by its ability to combine multiple connection characteristics when classifying network activity. However, the performance of a neural network depends on factors such as its architecture, data preprocessing, optimization settings, and the dataset used for evaluation. Research on intrusion detection has also highlighted the importance of carefully evaluating benchmark datasets and considering the differences between experimental conditions and operational networks [1]–[4].

This project investigates an MLP-based classifier for binary network anomaly detection using the NSL-KDD dataset. The proposed architecture contains three hidden layers with 64, 32, and 16 neurons, respectively, and uses ReLU activation in the hidden layers. The model is intended to use the Adam optimizer and a documented learning-rate strategy. The KDDTrain+ partition is used for model training, while KDDTest+ is reserved for final evaluation. Network connections are classified into two categories: normal and attack.

The study evaluates classification performance using accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix. A baseline classifier, such as logistic regression, is evaluated using the same data split and compatible preprocessing to provide a meaningful comparison. The study aims to determine how the proposed MLP performs on this benchmark, analyze its classification errors, and examine whether its performance provides a practical advantage over the selected baseline. The results are intended to support a reproducible comparison rather than establish that the proposed model is universally superior for real-world network security.

5. Research gap, motivation, and research questions

5.1 What is a research gap?

A research gap is an unresolved question, limitation, or underexplored comparison in existing research. It must be supported by reviewing previous papers; it should not simply be an assertion that nobody has ever used an MLP for intrusion detection.

Research already exists on neural networks, deep learning, and intrusion detection with NSL-KDD. Therefore, do not claim that MLP-based intrusion detection is new.

A more defensible gap for your project is the need for a clearly documented, reproducible comparison of a specified MLP configuration and a simpler baseline, using the same binary-label mapping, fixed training and test partitions, and multiple evaluation metrics. Whether the exact 64–32–16 architecture with the intended learning-rate schedule has already been studied needs to be verified by your literature search.

5.2 Proposed research-gap statement

Although machine-learning and neural-network methods have been widely investigated for network intrusion detection, reported results can be difficult to compare when studies use different datasets, preprocessing procedures, label definitions, model configurations, and evaluation metrics. NSL-KDD remains a useful benchmark for controlled experiments, but performance on this older dataset does not necessarily establish effectiveness on contemporary network traffic.

This project addresses a focused experimental question: how does a specified MLP architecture with hidden layers of 64, 32, and 16 neurons perform on binary normal-versus-attack classification using the NSL-KDD KDDTrain+ and KDDTest+ partitions, compared with a conventional baseline classifier? The study also examines classification errors, training-loss behavior, and the role of the selected optimization schedule. Its contribution is a reproducible, controlled comparison of the selected configurations, rather than a claim that MLPs or the dataset itself are new.

5.3 Research questions

Your team can use these four questions:

1. RQ1: How accurately can the proposed MLP distinguish normal network connections from attack connections in the NSL-KDD test set?
2. RQ2: How does the MLP compare with a logistic regression or decision-tree baseline in accuracy, precision, recall, F1-score, and ROC-AUC?
3. RQ3: What types of classification errors does the MLP make, especially false negatives in which attacks are classified as normal?
4. RQ4: How does the training-loss curve change over epochs under the selected learning-rate strategy, and what does it suggest about convergence or possible overfitting?

RQ4 requires an important qualification: if the team wants to claim that learning-rate decay improves performance, it needs a controlled comparison with an otherwise identical fixed-learning-rate model. A loss curve from a single run cannot prove that the decay schedule is better.

6. Experimental concepts you should understand

Even though Member 3 will implement the model, you should understand the metrics well enough to explain the results in the paper.

Metric	Meaning	Why it matters for intrusion detection
Accuracy	Proportion of all predictions that are correct	Gives an overall summary but can conceal poor detection of less frequent classes
Precision	Of all connections predicted as attacks, the proportion that really are attacks	Low precision means more false alarms
Recall	Of all actual attacks, the proportion detected by the model	Low recall means more attacks are missed
F1-score	Harmonic mean of precision and recall	Balances the two measures
ROC-AUC	Measures how well the model's scores rank attack examples above normal examples across thresholds	Evaluates ranking ability rather than only predictions at one threshold

Confusion matrix	Counts correct and incorrect predictions for each class	Shows false positives and false negatives directly

Assuming attack is the positive class, the main formulas are:

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

$$Precision = \frac{TP}{TP+FP}$$

$$Recall = \frac{TP}{TP+FN}$$

\[F1=2\cdot\frac{\text{Precision}\cdot\text{Recall}} {\text{Precision}+\text{Recall}} \]

Here:

- TP: attacks correctly predicted as attacks.
- TN: normal connections correctly predicted as normal.
- FP: normal connections incorrectly predicted as attacks.
- FN: attacks incorrectly predicted as normal.

For network security, false negatives can be particularly important because an attack may pass undetected. False positives matter too because too many incorrect alerts can burden security analysts.

The ROC-AUC score should be calculated using prediction scores or probabilities, not just hard class predictions. The team should also specify the positive class consistently.

What should the loss curve show?

A loss curve plots the model's loss against training epochs or iterations.

- A steadily decreasing training loss can indicate that the model is learning the training data.
- A loss that stops improving can indicate convergence or a training limitation.
- A decreasing training loss alongside worsening validation performance can indicate overfitting.

For `sklearn.neural_network.MLPClassifier`, `loss_curve_` records loss values across training iterations. The team should describe the x-axis as iterations or epochs according to how training is actually configured.



scikit-learn 1.8.0 documentation

7. Methodological points Member 1 should check with the team

These details are important because a paper can sound convincing while accidentally describing an experiment that was not implemented correctly.

- Feature preprocessing: The categorical fields `protocol_type`, `service`, and `flag` need encoding. Numeric features may need scaling. Preprocessing must be fitted using training data only and then applied to the test data.
- Binary labels: All attack types should be mapped to one attack class consistently in both files.
- Baseline fairness: The MLP and baseline should use the same training/test partition and compatible preprocessing. Any hyperparameter selection should use training data or a separate validation split.
- Learning-rate schedule: Standard scikit-learn `MLPClassifier` supports Adam, but its built-in learning-rate schedule parameter does not provide a custom epoch-wise Adam schedule. The paper must explain the actual implementation rather than imply unsupported functionality.
- Training and validation: Use validation data to select settings and monitor overfitting. Keep KDDTest+ reserved for final evaluation.
- Reproducibility: Record the random seed, preprocessing steps, architecture, optimizer settings, learning-rate schedule, stopping criteria, and software versions.
- Results: Report measured values from the actual runs. Do not copy performance numbers from other papers and present them as your model's results.

8. Future work

You can propose these directions in the final paper, as long as they are described as future work rather than completed experiments.

1. Evaluate the trained model on a newer intrusion-detection dataset, such as UNSW-NB15.
2. Investigate multiclass classification to distinguish DoS, Probe, R2L, and U2R attack families.
3. Compare different hidden-layer sizes and learning-rate schedules.
4. Study class-specific recall and false-positive rates, particularly for difficult or infrequent attacks.
5. Explore model interpretability so security analysts can better understand why a connection is flagged.
6. Investigate the computational cost and inference latency before making deployment claims.

Newer datasets such as UNSW-NB15 were developed partly because the field needs benchmarks that go beyond older intrusion datasets.

9. References in an ACM-style format

These are credible starting references for your research. Confirm the final bibliographic details against the publisher or original paper before submitting. Use the exact ACM reference style required by your instructor or template.

Foundational research and dataset references

1. Dorothy E. Denning. 1987. An intrusion-detection model. IEEE Transactions on Software Engineering 13, 2 (1987), 222–232. https://doi.org/10.1109/TSE.1987.232894

2. John McHugh. 2000. Testing intrusion detection systems: A critique of the 1998 and 1999 DARPA intrusion detection system evaluations. ACM Transactions on Information and System Security 3, 4 (2000), 262–294. https://doi.org/10.1145/382912.382923

3. Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. 2009. A detailed analysis of the KDD CUP 99 data set. In Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA). IEEE, 1–6. https://doi.org/10.1109/CISDA.2009.5356528

4. Robin Sommer and Vern Paxson. 2010. Outside the closed world: On using machine learning for network intrusion detection. In Proceedings of the IEEE Symposium on Security and Privacy. IEEE, 305–316. https://doi.org/10.1109/SP.2010.25

5. Animesh Patcha and Jung-Min Park. 2007. An overview of anomaly detection techniques: Existing solutions and latest technological trends. Computer Networks 51, 12 (2007), 3448–3470. https://doi.org/10.1016/j.comnet.2007.02.001

Machine learning and intrusion-detection references

6. Nour Moustafa and Jill Slay. 2015. UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In Proceedings of the Military Communications and Information Systems Conference (MilCIS). IEEE, 1–6. https://doi.org/10.1109/MilCIS.2015.7348942

7. Alisa L. Buczak and Erhan Guven. 2016. A survey of data mining and machine learning methods for cyber security intrusion detection. IEEE Communications Surveys & Tutorials 18, 2 (2016), 1153–1176. https://doi.org/10.1109/COMST.2015.2494502

8. Chi-Wen?

For the final reference list, use verified records for the additional neural-network papers you decide to cite. These are particularly relevant:

- N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi. 2018. A deep learning approach to network intrusion detection. IEEE Transactions on Emerging Topics in Computational Intelligence 2, 1 (2018), 41–50. https://doi.org/10.1109/TETCI.2017.2772792
- C. Yin, Y. Zhu, J. Fei, and X. He. 2017. A deep learning approach for intrusion detection using recurrent neural networks. IEEE Access 5 (2017), 21954–21961.
- A survey of neural networks usage for intrusion detection systems. 2021. Journal of Ambient Intelligence and Humanized Computing. https://doi.org/10.1007/s12652-020-02014-x
- Survey of intrusion detection systems: techniques, datasets and challenges. 2019. Cybersecurity. https://doi.org/10.1186/s42400-019-0038-7

The sources above cover intrusion-detection fundamentals, evaluation methodology, dataset quality, anomaly detection, neural-network methods, and alternative benchmarks.

Sanavi Content

3. Dataset Description

3.1 NSL-KDD Dataset

The NSL-KDD dataset is a benchmark dataset widely used in research on network intrusion detection systems (IDS). It was developed to address several limitations of the original KDD Cup 1999 dataset, including redundant records and issues with the distribution of training and testing instances. These improvements make NSL-KDD useful for evaluating and comparing supervised machine-learning approaches for intrusion detection.

The dataset consists of labelled network connection records, with each record represented by 41 input features describing connection characteristics, traffic behavior, and selected content-related indicators. In the standard dataset format, each record also includes an attack label and a difficulty-level field.

The dataset provides separate training and testing files, namely `KDDTrain+` and `KDDTest+`. The standard training set contains 125,973 records, while the standard test set contains 22,544 records. These counts should be verified against the files used in the experiments.

The input features include basic connection information, such as protocol type, service, connection status, duration, and bytes transferred. Additional features describe login behavior, suspicious activities, and statistical patterns of connections to particular hosts and services.

NSL-KDD includes normal network traffic and several attack categories, conventionally grouped into Denial of Service (DoS), Probe, Remote-to-Local (R2L), and User-to-Root (U2R). The distribution of records across these classes is imbalanced, with some attack categories represented by substantially fewer examples than others.

Dataset source: University of New Brunswick, Canadian Institute for Cybersecurity, NSL-KDD dataset documentation: https://www.unb.ca/cic/datasets/nsl.html

3.2 Problem Formulation

This study considers network anomaly detection as a supervised binary classification problem. Each network connection is classified into one of two categories: normal traffic or attack traffic.

The original normal label is mapped to class 0, while all attack labels are mapped to class 1. This formulation enables the proposed Multilayer Perceptron (MLP) classifier to distinguish potentially malicious network connections from legitimate activity.

The difficulty-level field is excluded from the input features because it is metadata rather than one of the 41 original network-connection features. The classifier uses the network features to learn patterns associated with normal and anomalous connections.

4. Data Preprocessing

4.1 Data Inspection and Cleaning

Data preprocessing is performed before training the MLP classifier to ensure that the input features are consistent and suitable for numerical computation. The training and testing files are loaded separately, and the documented feature names are assigned to the corresponding columns.

The initial inspection examines dataset dimensions, feature data types, missing values, duplicate records, and class-label distributions. Any missing, malformed, or invalid values identified during inspection are investigated and handled using a documented procedure. Records are not removed automatically without justification, since unnecessary deletion may alter the dataset distribution and affect the evaluation.

The original training and testing partitions are preserved throughout preprocessing. This approach maintains a clear distinction between model development and final evaluation.

4.2 Categorical Feature Encoding

The NSL-KDD dataset contains three categorical input features: `protocol_type`, `service`, and `flag`. Since these features contain nominal categories rather than continuous numerical measurements, they must be transformed into numerical representations before being supplied to the MLP classifier.

One-hot encoding is used to represent each categorical value through binary indicator features. This method avoids introducing an artificial numerical ordering among categories and allows the classifier to learn relationships between categorical connection characteristics and traffic labels.

The encoder is fitted using the training data and then applied to the test data. Previously unseen test categories are handled using an encoding configuration that ignores unknown categories, ensuring consistent feature dimensions between the two datasets.

4.3 Binary Label Encoding

The original NSL-KDD labels represent normal connections and different attack types. For the binary classification task, these labels are converted into two classes:

- Normal traffic: 0
- Attack traffic: 1

All attack categories are combined into a single positive class. The original labels are retained separately for dataset analysis and reporting.

This transformation aligns the dataset with the objective of detecting anomalous network connections rather than identifying individual attack types.

4.4 Numerical Feature Scaling

Numerical input features in NSL-KDD have different ranges and units. For example, byte counts and connection durations can have substantially larger numerical magnitudes than rate-based features.

To reduce the influence of differences in feature scale, standardization is performed using `StandardScaler`. Each numerical feature is transformed according to the mean and standard deviation estimated from the training data:

$z = (x - \mu_{train}) / \sigma_{train}$

where x represents the original feature value, μ_{train} represents the training-set mean, and σ_{train} represents the training-set standard deviation.

The scaler is fitted exclusively on the training data and subsequently applied to both the training and test data. This prevents information from the test set from influencing the preprocessing parameters and reduces the risk of data leakage.

4.5 Class Distribution and Imbalance

Class distribution is examined before and after converting the original labels into binary classes. This analysis determines the number and proportion of normal and attack records in each dataset partition.

Class imbalance is an important consideration in intrusion detection because a classifier may achieve high overall accuracy while failing to detect a meaningful proportion of attacks. Therefore, the class distribution is documented, and model performance is intended to be evaluated using precision, recall, F1-score, and a confusion matrix in addition to accuracy.

Any class-balancing procedure, if used, must be explicitly documented and restricted to the training data. The test distribution should remain unchanged to provide a consistent evaluation of the trained model.

4.6 Final Feature Representation

After categorical encoding and numerical scaling, the preprocessing pipeline produces numerical feature matrices suitable for the proposed MLP classifier. The target labels are maintained separately from the input features.

The final number of input features is determined by the one-hot encoding configuration and the categories observed during fitting. Consequently, the encoded feature count may exceed the original 41 input features.

The resulting training and test matrices preserve the same feature order and dimensionality. These processed matrices are supplied to the MLP classifier, which uses a hidden-layer architecture of 64, 32, and 16 neurons, followed by the output layer.

The preprocessing pipeline and its fitted transformations are retained to support reproducibility and consistent processing of subsequent network-connection records.

Table 1. Dataset summary

Parameter	Training set	Test set
Total records	125,973*	22,544*
Original input features	41	41
Normal records	67,343*	9,711*
Attack records	58,630*	12,833*
Classification task	Binary	Binary

Table 2. Feature preprocessing summary

Feature group	Original representation	Preprocessing method
Numerical features	38 numerical/binary features	StandardScaler
`protocol_type`	Categorical	One-hot encoding
`service`	Categorical	One-hot encoding
`flag`	Categorical	One-hot encoding
Original attack label	Multiple classes	Binary label encoding

Difficulty field | Dataset metadata | Excluded from model inputs